

SERSEM: Selective Entropy-Weighted Scoring for Membership Inference in Code Language Models

Kıvanç Kuzey Dikici*

Bilkent University
Computer Engineering
Ankara, Turkey
kuzey.dikici@ug.bilkent.edu.tr

Serdar Kara*

Bilkent University
Computer Engineering
Ankara, Turkey
serdar.kara@ug.bilkent.edu.tr

Semih Çağlar

Bilkent University
Computer Engineering
Ankara, Turkey
semih.caglar@ug.bilkent.edu.tr

Eray Tüzün

Bilkent University
Computer Engineering
Ankara, Turkey
eraytuzun@cs.bilkent.edu.tr

Sinem Sav

Bilkent University
Computer Engineering
Ankara, Turkey
sinem.sav@cs.bilkent.edu.tr

Abstract

As Large Language Models (LLMs) for code increasingly utilize massive, often non-permissively licensed datasets, evaluating data contamination through Membership Inference Attacks (MIAs) has become critical. We propose **SERSEM** (Selective Entropy-Weighted Scoring for Membership Inference), a novel white-box attack framework that suppresses uninformative syntactical boilerplate to amplify specific memorization signals. SERSEM utilizes a dual-signal **methodology**: first, a continuous character-level weight mask is derived through static Abstract Syntax Tree (AST) analysis, spellchecking-based multilingual logic detection, and offline linting. Second, these heuristic weights are used to pool internal transformer activations and calibrate token-level Z-scores from the output logits. Evaluated on a 25,000-sample balanced dataset, SERSEM achieves a global **AUC-ROC of 0.7913** on the StarCoder2-3B model and **0.7867** on the StarCoder2-7B model, consistently outperforming the implemented probability-based baselines Loss, Min-K% Prob, and PAC. Our findings demonstrate that focusing on human-centric coding anomalies provides a significantly more robust indicator of verbatim memorization than sequence-level probability averages.

CCS Concepts

• **Security and privacy** → **Domain-specific security and privacy architectures**; • **Software and its engineering**;

Keywords

Large Language Models, Privacy, Memorization, Membership Inference, Data Leakage, Selective Entropy Scoring

ACM Reference Format:

Kıvanç Kuzey Dikici, Serdar Kara, Semih Çağlar, Eray Tüzün, and Sinem Sav. 2026. SERSEM: Selective Entropy-Weighted Scoring for Membership Inference in Code Language Models. In *34th ACM Joint European Software*

*Equal contribution with alphabetical order.



This work is licensed under a Creative Commons Attribution 4.0 International License. FSE Companion '26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2636-1/2026/07
<https://doi.org/10.1145/3803437.3807734>

Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion '26), July 05–09, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3803437.3807734>

1 Introduction

Large Language Models (LLMs) trained on source code have demonstrated remarkable capabilities in code generation, completion, and reasoning [11]. Nonetheless, since these models scale in size and are exposed to massive corpora such as The Stack v2 [9], they increasingly exhibit a tendency to memorize portions of their training data [1]. This memorization poses significant legal and ethical risks, particularly when models inadvertently reproduce non-permissively licensed or proprietary code snippets such as those curated in The Heap dataset [7].

To assess the extent of data contamination in closed-data models, researchers rely on Membership Inference Attacks (MIAs) [13]. In a general white-box membership inference setting, the adversary attempts to determine whether a specific data point was included in the target model's training set by computing probabilities over the target model [3, 10, 13]. Although standard baselines such as negative log-likelihood (Loss), Min-K % Prob [12], Surprise-based MIAs (SURP) [15], and Polarized Augment Calibration (PAC) [14] have been proposed, they fundamentally treat source code as a natural language. Source code, however, is structurally bound by rigid syntax (e.g., for, def, return). Because an LLM can predict standard boilerplate with near-perfect confidence regardless of whether it has memorized the specific file, these probability techniques suffer from high false-positive rates when applied to software artifacts [12, 15].

The Poisoned Chalice Competition 2026 [6] establishes a white-box evaluation framework to advance domain-specific MIAs against code models. In this paper, we present our solution: **SERSEM** (Selective Entropy-Weighted Scoring for Membership Inference). SERSEM shifts the attack vector from general sequence prediction to continuous targeted anomaly detection. We assert that prediction of human fallibility, such as formatting inconsistencies, emotional comments, multilingual logic slippage, and idiosyncratic naming conventions, can provide a useful signal of verbatim memorization.

Furthermore, our approach aligns the heuristic coding weights with deep internal activation profiles. By extracting membership

signals from the intermediate layers of the transformer rather than relying exclusively on final output logits, we isolate the representational signatures of memorization before the model optimizes for syntactical correctness [5]. By continuously weighting these internal and output probability distributions based on the entropic density of the code segment, our approach suppresses boilerplate noise and improves MIA accuracy against data-blurred code models.

2 Methodology

This section details the design and implementation of the SERSEM framework, an attack specifically engineered to exploit the fine-grained distinctions of memorized source code. The core architecture of SERSEM utilizes a dual-angled approach (see Figure 1). First, it extracts human-centric anomalies through static code analysis to construct a continuous character-level weight mask. Second, it utilizes both the final output logits and the deeply embedded transformer activations to derive a final probability score.

2.1 Continuous Entropy-Weighted Scoring

Standard source code comprises highly predictable repeating boilerplate syntax. When these tokens are generated, standard causal LMs yield extremely high probabilities regardless of memorization, making the genuine membership signal thinner [12, 15].

To counteract this, WeightedAnomalyMIA assigns a specific continuous weight to each token in the evaluation sequence, prioritizing tokens that reflect human irregularity. We construct a custom anomaly map by extracting six critical continuous signals via an Abstract Syntax Tree (AST) using tree-sitter [2] and secondary heuristics:

- **Boilerplate Noise Mitigation:** To suppress the noise of predictable syntax and punctuation, tokens failing a structural identifier regex match (e.g., brackets, operators, keywords, short auxiliary variables) are penalized and downgraded to a negligible weight.
- **Standard Identifiers:** Code tokens that strictly match valid programmable variable formatting are preserved at a baseline standard weight.
- **Long Identifiers:** To specifically prioritize developer-centric naming, identifiers exceeding 10 characters are assigned an elevated weight by the AST.
- **Literal Strings:** AST parsing dynamically inflates weights for high-entropy data artifacts such as string literals.
- **Linting Formatting Errors:** Offline linting maps abnormal spacing, inconsistent indentation, and arbitrary structural chaos to absolute character offsets. For Python, this is executed via flake8 [4]; other languages utilize generic regex fallbacks.
- **Comments & Multilingual Slippage:** Standard comments mapped via the AST, and identifiers failing an en_US PyEnchant [8] dictionary check, receive a severe penalty for demonstrating out-of-distribution human blunders.
- **Psychological Tags:** An override regex specifically targets high-confidence developer signatures (e.g., TODO, FIXME).

During inference, these character-level heuristic signals are projected onto the subword tokens generated by the tokenizer. The

final consolidated token weights that drive the anomaly scoring mechanism are summarized in Table 1.

Table 1: SERSEM Final Token Weighting Scheme.

Token Classification	Weight
Boilerplate Noise & Syntax	0.1
Standard Identifiers (< 10 chars)	1.0
Long Identifiers (≥ 10 chars)	3.0
Literal Strings	5.0
Linting Formatting Errors	5.0
Comments & Multilingual Slippage	10.0
Psychological Tags (TODO)	10.0

Instead of evaluating raw output probabilities, SERSEM computes the **Z-Score** of the correct next token's logit relative to the full vocabulary distribution at that prediction step. By passing the Z-Scores through a sigmoid layer and multiplying them against the normalized token weight array, the final score inherently grades the LLM exclusively on how good it predicts human blind spots while suppressing structural boilerplate.

2.2 Internal Activation Probing

Although the output logits provide significant predictive insight, the final layers of a transformer are biased towards syntactical generation [5]. Verbatim memorization signatures are typically strongest within the hidden states of intermediate layers. To capture these deep representational signals, we integrate LUMIA (Linear probing for Unimodal and Multimodal Membership Inference Attacks leveraging internal LLM states) [5]. The core assertion of LUMIA is that the internal parameter activations inside a white-box language model preserve distinguishable characteristics of its training data before they are smoothed out by the final classification head.

Leveraging the white-box constraints of the Poisoned Chalice Competition [6], SERSEM attaches dynamic forward hooks to every transformer layer of the target model. We extract the internal activations during a single, no-gradient forward pass. Because pure mean-pooling risks flattening the signal of rare keywords, we independently pool the activations using both standard averaging and the entropy-based structural weights described in Section 2.1. These representations are concatenated to yield an augmented activation space.

Following the LUMIA methodology [5], we evaluate a random subset of validation shadow data to extract layer activations for known members and non-members. These extracted features allow us to dynamically select the most discriminative hidden layers. A lightweight Multi-Layer Perceptron (MLP) probe is trained via binary cross-entropy on the concatenated internal representations for each chosen layer to predict binary membership. In inference, the attack dynamically ensembles the sigmoid outputs from the top-performing five layer probes, outputting a precise membership 230 confidence score bounded between [0.0, 1.0].

3 Experiments and Results

This section presents the empirical evaluation of SERSEM, covering the experimental setup, the performance of the per-language

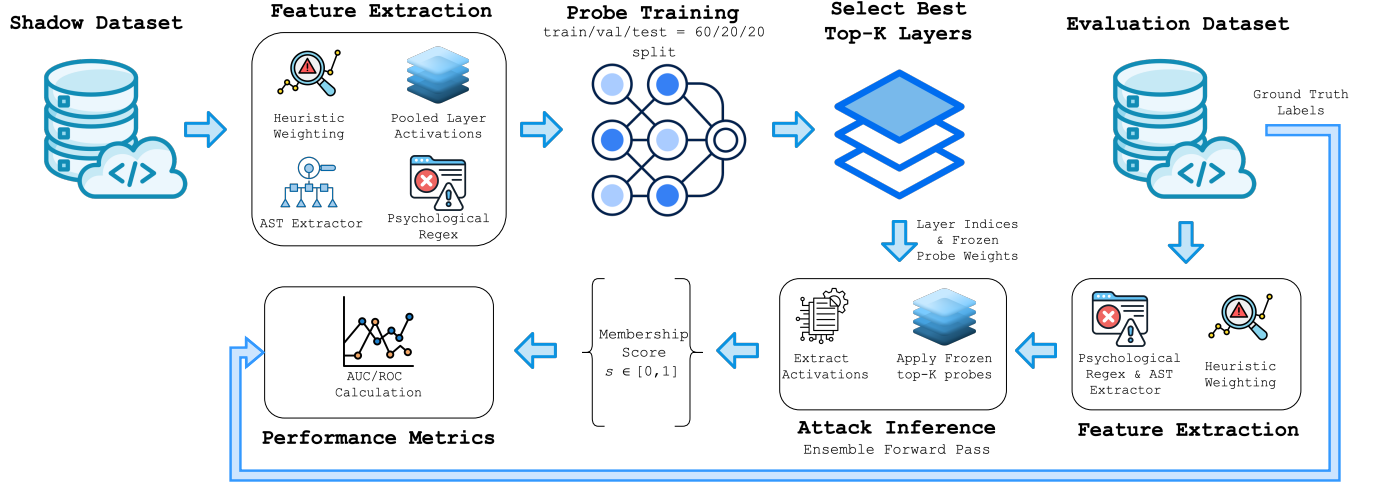


Figure 1: Architecture and workflow of the SERSEM framework.

membership inference, and a discussion of the detection variability across programming paradigms and baseline comparisons. The AUC-ROC curves for both target models are illustrated in Figure 2.

3.1 Experimental Setup

To conform to the competition structure while addressing the lack of a ground-truth held-out test set during validation, we partitioned the provided Poisoned Chalice development dataset [6]. For each of the five target languages (Python, Java, Go, Ruby, Rust), we extracted a random sampling of 10,000 instances **balanced equally** between members (The Stack v2 [11]) and non-members (The Heap [7]).

From this pool, 5,000 samples per language (25,000 total) were allocated for training our intermediate layer probe-training¹. To ensure the complete integrity of our Results section and the final competition submission, we enforce two separate layers of data isolation. First, we maintain a no-overlap property between our internal training and inference splits (5,000 samples each per language). Second, we have confirmed with the organizers that the official held-out evaluation set used for final scoring has zero intersection with the released development data.

The SERSEM pipeline was developed and validated on a high-performance workstation featuring a single NVIDIA TITAN RTX (24GB VRAM) GPU, dual Intel(R) Xeon(R) Gold 6140 CPUs (36 physical cores, 72 threads), 256GB of system RAM, and a 2.2TB high-speed data partition. We evaluated our approach on the **StarCoder2-3B** and **StarCoder2-7B** models [11], which were pre-trained on **The Stack v2** dataset [11]. This ensures that samples drawn from The Stack v2 are verifiably members of the model’s training corpus. Due to the architectural **limitations** of our local validation environment (TITAN RTX), all experiments reported in this paper were conducted utilizing float16 precision and sequence lengths of up to 8,196 tokens. However, our proposed submission pipeline is fully compatible with bfloat16 precision for the final evaluation

Table 2: Per-language AUC-ROC comparison of SERSEM and implemented baselines on 5,000 samples per language.

Language	SERSEM 3B	SERSEM 7B	Loss 3B	MKP 3B	PAC 3B
Overall	0.7913	0.7867	0.5881	0.5931	0.4712
Go	0.8121	0.8071	0.5978	0.6016	0.5023
Java	0.7738	0.7632	0.5856	0.5906	0.4637
Python	0.8066	0.8045	0.6027	0.6071	0.4700
Ruby	0.8155	0.8151	0.6302	0.6367	0.4237
Rust	0.7402	0.7379	0.5643	0.5658	0.4898

on the competition’s NVIDIA A100 hardware, ensuring optimal performance within the 12-hour wall-clock limit.

3.2 Empirical Analysis of Detection Accuracies

Evaluating our full pipeline on held-out inference splits of 5,000 samples per language, SERSEM achieves an overall AUC-ROC of **0.7913** on StarCoder2-3B and **0.7867** on StarCoder2-7B (Figure 2), indicating strong membership detection performance. The Loss, Min-K% Prob, and PAC baselines in Table 2 were run using the official PoisonedChalice implementations²; due to local hardware constraints, PAC was executed in float16 rather than bfloat16, consistent with our local validation setup.

To provide granular insight into the attack’s effectiveness across different syntax structures, a detailed breakdown of the AUC-ROC performance per programming language is presented in Table 2.

3.3 Discussion of Language Variability

The results highlight a significant divergence in detection performance between programming paradigms. Ruby (0.8155) and Go (0.8121) achieved the highest AUC scores for the 3B model, while Ruby (0.8151) remained the language that performed the best for the 7B variant. These high scores are likely due to the flexible, highly idiosyncratic formatting rules and human conventions in

¹The trained shadow model weights are publicly available on HuggingFace: <https://huggingface.co/Serdark4r/SERSEM-Poisoned-Chalice-Competition-2026>

²<https://github.com/AISE-TUdelft/PoisonedChalice>

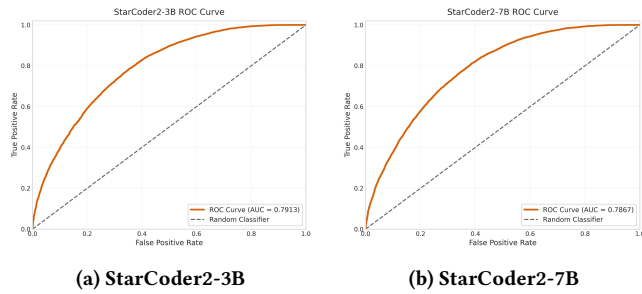


Figure 2: SERSEM AUC-ROC curves.

these languages, which allow for greater expression of human coding chaos. Python also exhibited consistently strong performance across both model sizes (0.8066 for 3B and 0.8045 for 7B), suggesting that although Python has a more standardized and readable syntax than Ruby, it still preserves substantial room for human-specific variation through comments, naming styles, spacing behavior, and informal scripting conventions. In this sense, Python appears to occupy a middle ground: it is less structurally rigid than Java and Rust, but less stylistically unconstrained than Ruby, making it particularly suitable for SERSEM’s anomaly-focused weighting strategy. Conversely, Rust (0.7402) and Java (0.7738) enforce stricter structural and typed boilerplate, which inherently suppress unique human variations and reduce the impact of the static continuous penalty mask.

3.4 Discussion of Baseline Comparisons

As shown in Table 2, SERSEM consistently outperforms all three baselines implemented on the StarCoder2-3B model. Loss, Min-K% Prob (MKP), and PAC achieve overall AUC-ROC values of 0.5881, 0.5931, and 0.4712, respectively, whereas SERSEM reaches 0.7913, representing an improvement of more than 20 percentage points. This margin is consistent across all five languages: even in Rust, where SERSEM attains its lowest AUC (0.7402), it remains substantially above the strongest baseline. This performance gap reflects a limitation shared by all three baselines: they rely exclusively on output-probability information and remain agnostic to the structural role each token plays in source code. Loss aggregates token-level likelihood across the sequence, PAC calibrates sequence-level confidence using augmented neighboring samples, and even the more selective Min-K% Prob focuses only on the least likely tokens without incorporating code structure. As a result, these methods cannot distinguish between tokens that are difficult because they reflect genuine training data exposure and tokens that are simply rare under the model’s general distribution. SERSEM addresses this limitation by grounding its weighting scheme in code structure.

4 Conclusion

In this paper, we present **SERSEM**, a white-box membership inference attack designed for code language models. By shifting from global probability distributions to structure-aware anomaly detection, SERSEM isolates human-centric memorization signals in source code. Our method combines static AST-based heuristics and automated linting with internal activation probes, allowing

it to leverage deeper representational signals that are often obscured in the final output logits. Evaluated on a 25,000-sample dataset, SERSEM achieved a robust AUC-ROC of **0.7913** against StarCoder2-3B and **0.7867** against StarCoder2-7B. It also consistently outperforms the implemented probability-based baselines Loss, Min-K% Prob, and PAC on our held-out inference split. Future work includes extending entropy-weighted masks to broader language paradigms, automatically tuning heuristic weights per target distribution, and replacing regex-based fallbacks with native linters for non-Python languages. This paper is submitted as part of the *Poisoned Chalice Competition* [6], and the replication package for this work is publicly available³.

References

- [1] Ali Al-Kaswan, Maliheh Izadi, and Arie van Deursen. 2024. Traces of Memorisation in Large Language Models for Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
- [2] Max Brunsfeld et al. 2024. Tree-sitter: An Incremental Parsing System for Programming Tools. <https://tree-sitter.github.io/tree-sitter/> Accessed: 2026.
- [3] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. 2022. Quantifying memorization across neural language models. In *The Eleventh International Conference on Learning Representations*. *arXiv preprint arXiv:2202.07646*.
- [4] Ian Cordasco et al. 2024. Flake8: Your Tool for Style Guide Enforcement. <https://flake8.pycqa.org/> Accessed: 2026.
- [5] Luis Ibanez-Lissen, Lorena Gonzalez-Manzano, Jose Maria de Fuentes, Nicolas Anciaux, and Joaquin Garcia-Alfaro. 2025. LUMIA: Linear Probing for Unimodal and MultiModal Membership Inference Attacks Leveraging Internal LLM States. In *Computer Security – ESORICS 2025: 30th European Symposium on Research in Computer Security, Toulouse, France, September 22–24, 2025, Proceedings, Part I* (Toulouse, France). Springer-Verlag, Berlin, Heidelberg, 186–206. doi:10.1007/978-3-032-07884-1_10
- [6] Jonathan Katzy, Ali Al-Kaswan, Razvan Mihai Popescu, and Zhou Yang. 2026. The Poisoned Chalice of LLM Evaluation Report. In *Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE ’26)*. ACM.
- [7] Jonathan Katzy, Razvan Mihai Popescu, Arie van Deursen, and Maliheh Izadi. 2025. The Heap: A Contamination-Free Multilingual Code Dataset for Evaluating Large Language Models. *arXiv:2501.09653* <https://arxiv.org/abs/2501.09653>
- [8] Ryan Kelly et al. 2024. PyEnchant: Spellchecking for Python. <https://pyenchant.github.io/pyenchant/> Accessed: 2026.
- [9] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. 2022. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533* (2022).
- [10] Chenxi Li, Abhinav Kumar, Zhen Guo, Jie Hou, and Reza Tourani. 2024. Unveiling the Unseen: Exploring Whitebox Membership Inference through the Lens of Explainability. *CoRR abs/2407.01306* (2024). *arXiv:2407.01306* doi:10.48550/ARXIV.2407.01306
- [11] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Gabriel Cassano, Joel Lamy-Poirier, Nouamane Tazi, Nikolay Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv preprint arXiv:2402.19173* (2024). <https://huggingface.co/bigcode/starcoder2-3b>
- [12] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2023. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789* (2023).
- [13] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership Inference Attacks Against Machine Learning Models. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22–26, 2017*. IEEE Computer Society, USA, 3–18. doi:10.1109/SP.2017.41
- [14] Wentao Ye, Jiaqi Hu, Liyao Li, Haobo Wang, Gang Chen, and Junbo Zhao. 2024. Data Contamination Calibration for Black-box LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand. <https://aclanthology.org/2024.findings-acl.644/>
- [15] Anqi Zhang and Chaofeng Wu. 2024. Adaptive pre-training data detection for large language models via surprising tokens. *arXiv preprint arXiv:2407.21248* (2024).

³<https://doi.org/10.6084/m9.figshare.31911933>